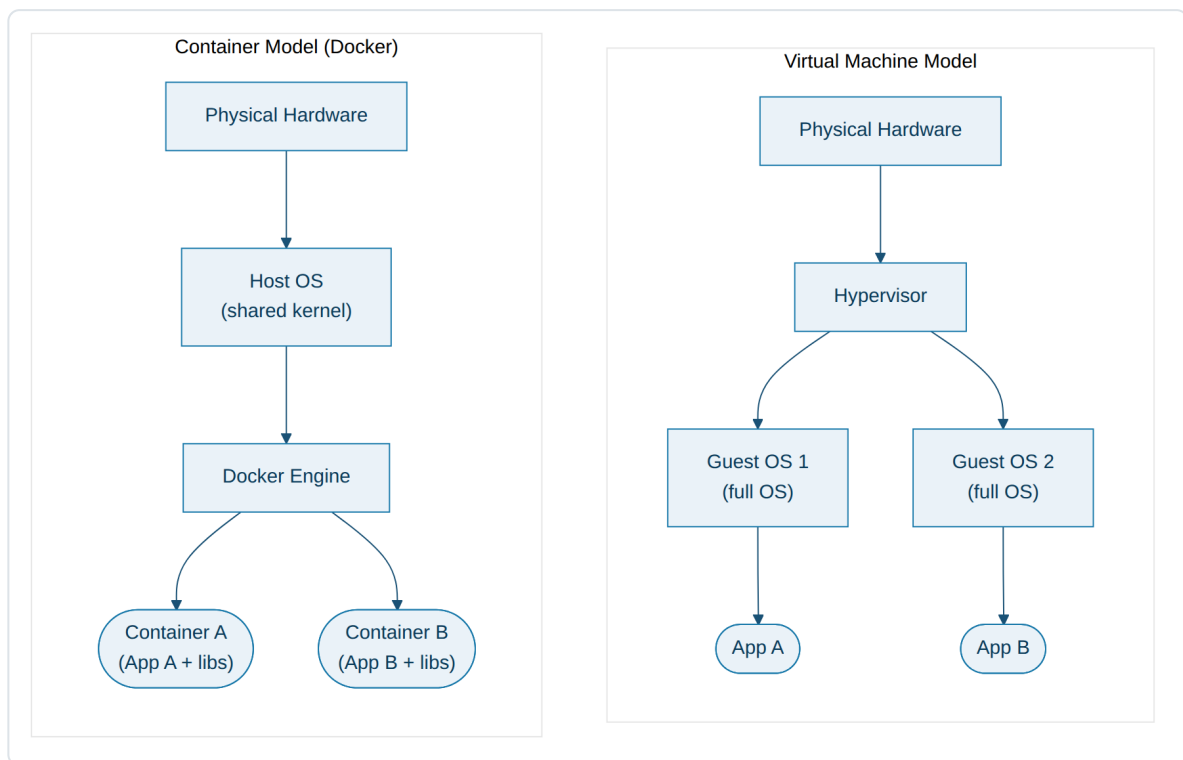


Kubernetes, Docker & GKE Notes

Why Containers? (Docker vs Virtual Machines)

- Before containers → apps were run on full Virtual Machines (VMs), each with its own complete OS (Guest OS) — heavy (GBs), slow to boot (minutes), wastes CPU/memory on duplicate OS overhead.
- Containers → package an app + its dependencies (libraries, runtime, config) together, but **share the host OS kernel** — lightweight (MBs), boot in seconds, many containers per host.
- Docker → the most popular container platform/tool → builds, ships, and runs containers.

Container vs VM diagram

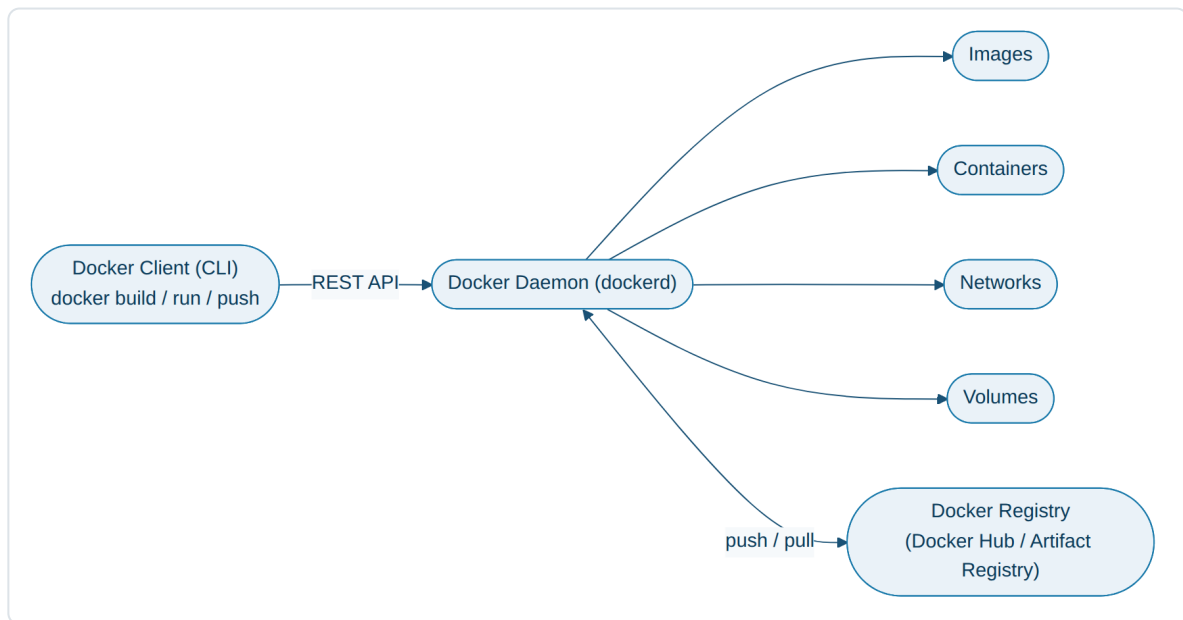


Docker Architecture

- Docker follows a **client-server** model:
- **Docker Client (CLI)** → the `docker` command you type (`docker build` , `docker run` , `docker push` ...).
- **Docker Daemon (`dockerd`)** → background service that does the real work — builds images, runs/stops containers, manages networks & volumes. Runs on the Docker Host.

- **Docker Registry** → a storage/distribution service for images (Docker Hub is the public default; Artifact Registry / Container Registry are GCP's private registries).
- The CLI talks to the daemon over a REST API (usually a local Unix socket).

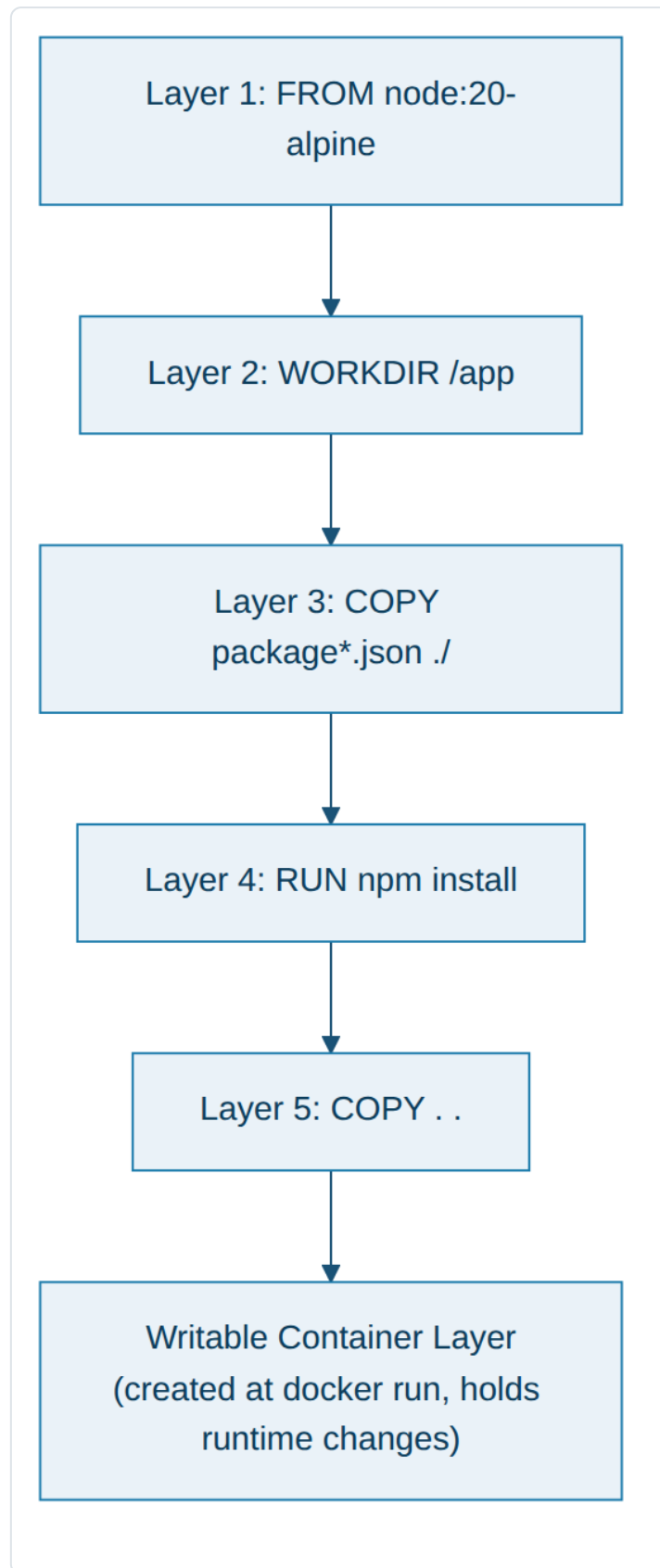
Docker architecture diagram



Docker Images

- **Docker Image** → a read-only template/blueprint containing the application code, runtime, libraries, environment variables, and a default startup command.
- **Docker Container** → a running (or stopped) **instance** of an image — the same relationship as a *class* (image) and an *object* (container). One image → many containers.
- Images are built from a **Dockerfile** — a text file of step-by-step instructions.
- Images are made of **layers** → each Dockerfile instruction (**FROM**, **RUN**, **COPY**, ...) creates a new read-only layer, stacked on top of each other. Layers are cached & reused across builds/images → faster builds, less disk usage.
- When a container runs, Docker adds one thin **writable layer** on top of the image's read-only layers — any file changes happen there; the underlying image never changes.

Docker image layers diagram



Common Dockerfile instructions

- **FROM** → base image to start from (eg. `node:20-alpine`).
- **WORKDIR** → sets the working directory inside the image.
- **COPY** / **ADD** → copies files from your machine into the image.
- **RUN** → executes a command *at build time* (eg. install dependencies) → creates a layer.
- **ENV** → sets environment variables.
- **EXPOSE** → documents which port the container listens on.
- **CMD** / **ENTRYPOINT** → the command that runs *when the container starts*.

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["npm", "start"]
```

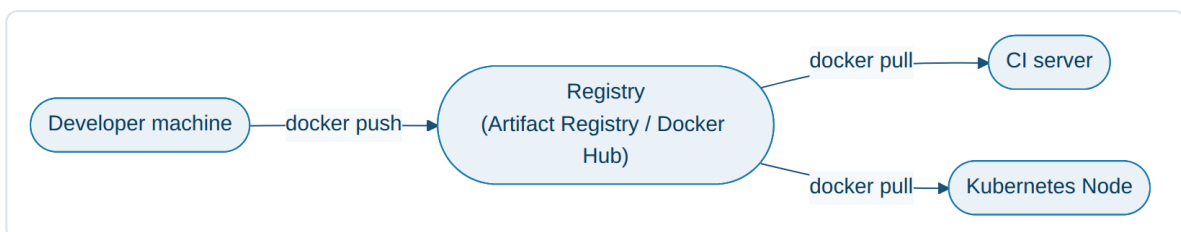
Build → Run flow



```
docker build -t my-node-app:1.0 .
docker run -d -p 3000:3000 --name my-app-container my-node-app:1.0
```

Registry → Push / Pull flow

- Images are tagged (`name:tag` , eg. `my-node-app:1.0`) and pushed to a registry so other machines/clusters can pull & run the exact same image.

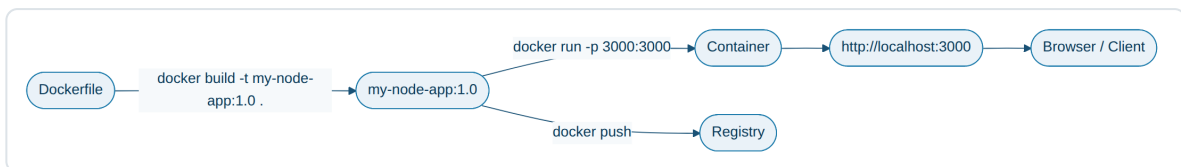


```
docker tag my-node-app:1.0 asia-south1-docker.pkg.dev/my-project/my-repo/my-node-app:1.0
docker push asia-south1-docker.pkg.dev/my-project/my-repo/my-node-app:1.0
```

Example → Dockerizing a Node.js web app

1. Write the app (`server.js` , `package.json`) that listens on port `3000` .

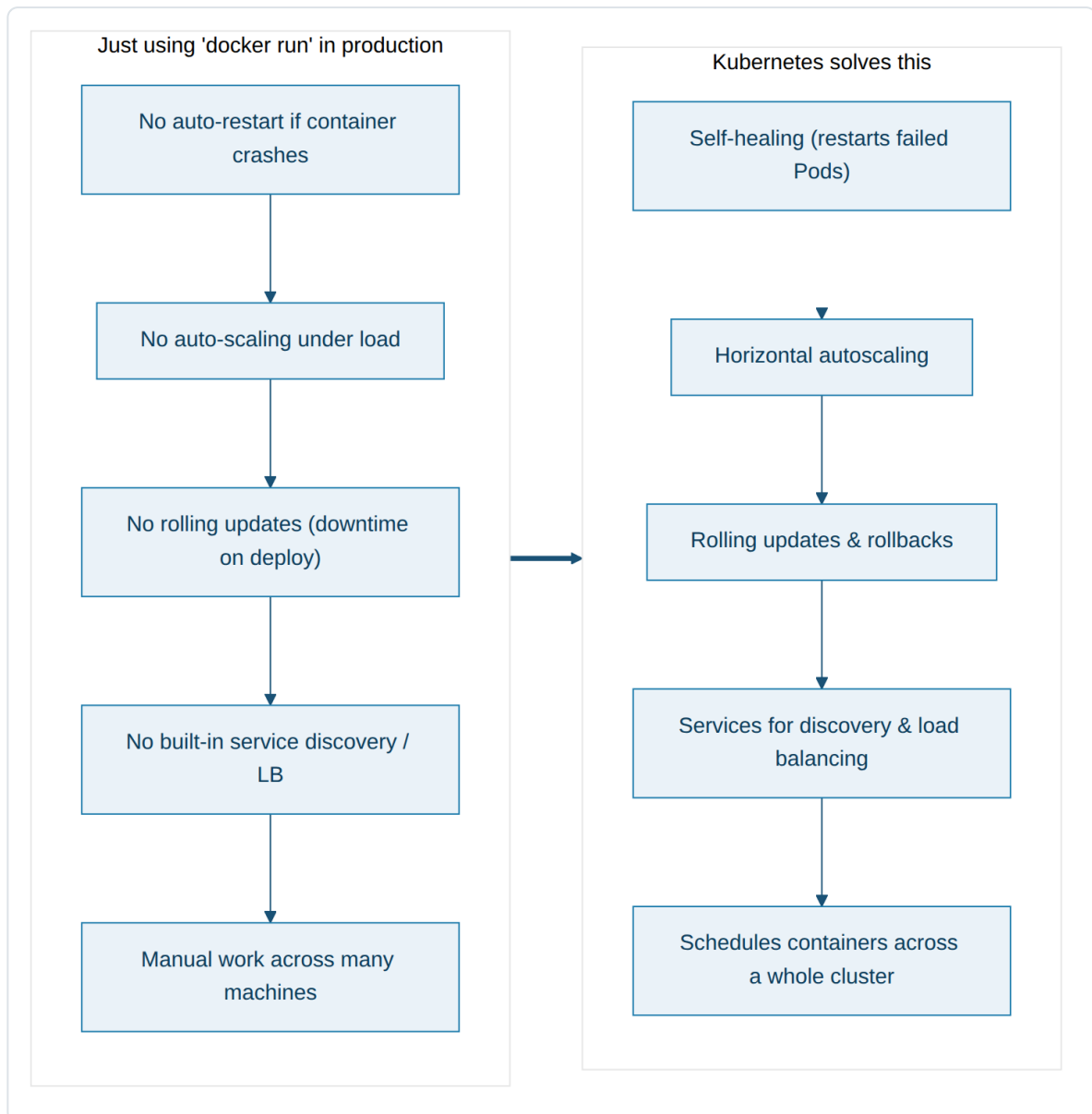
2. Write the `Dockerfile` above in the project root.
3. `docker build -t my-node-app:1.0 .` → Docker reads the Dockerfile top-to-bottom, creates a layer per instruction, produces the final image `my-node-app:1.0`.
4. `docker run -p 3000:3000 my-node-app:1.0` → Docker creates a container from the image, maps host port `3000` → container port `3000`.
5. Visit `http://localhost:3000` → request hits the container → Node.js app responds.
6. `docker push` the image to a registry so it can later be deployed to Kubernetes/GKE.



Why Kubernetes?

- Running a handful of containers manually with `docker run` works fine on one machine — but production needs:
 - Multiple containers across many machines (nodes)
 - Automatic restart if a container crashes
 - Scaling up/down based on load
 - Rolling out new versions with zero downtime
 - Service discovery & load balancing between containers
 - Self-healing or Node failures
- **Kubernetes (K8s)** → an open-source **container orchestration** platform that automates deployment, scaling, networking, and management of containerized applications across a cluster of machines.

The problem Kubernetes solves



Kubernetes Architecture

- A Kubernetes **Cluster** = one **Control Plane** (the "brain") + one or more **Worker Nodes** (where your containers actually run).

Control Plane components

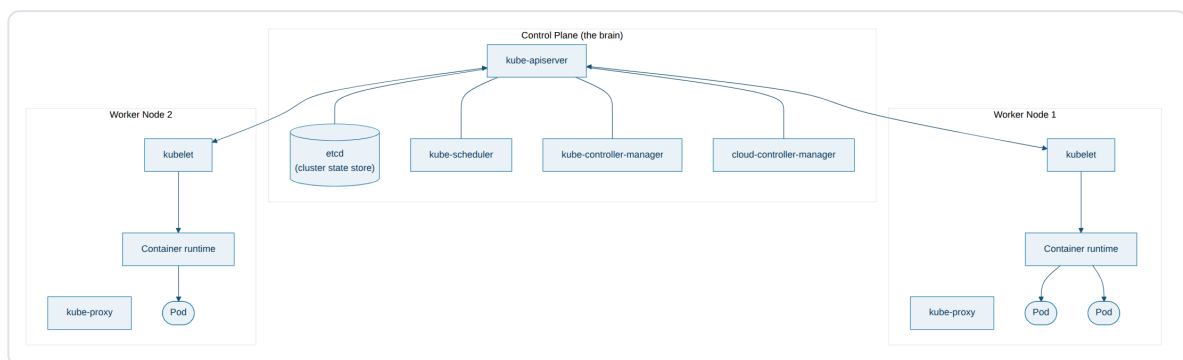
- **kube-apiserver** → the front door — every request (from **kubectl**, dashboards, controllers) goes through the API server; it validates & processes REST operations.
- **etcd** → a distributed, consistent key-value store — the cluster's single source of truth (all cluster state/config is stored here).

- **kube-scheduler** → watches for newly created Pods with no Node assigned, and picks the best Node for them (based on resources, affinity rules, taints/tolerations).
- **kube-controller-manager** → runs controller loops that continuously watch cluster state and reconcile it to match the desired state (eg. Node controller, ReplicaSet controller).
- **cloud-controller-manager** → talks to the underlying cloud provider (GCP/AWS/Azure) to manage cloud resources like load balancers & disks (this piece is Google-managed in GKE).

Worker Node components

- **kubelet** → the agent on every node — talks to the API server, ensures the containers described in a Pod spec are actually running & healthy.
- **kube-proxy** → maintains network rules on each node so Services can route traffic to the correct Pods.
- **Container runtime** (containerd, CRI-O) → the software that actually pulls images and runs containers.

Kubernetes architecture diagram

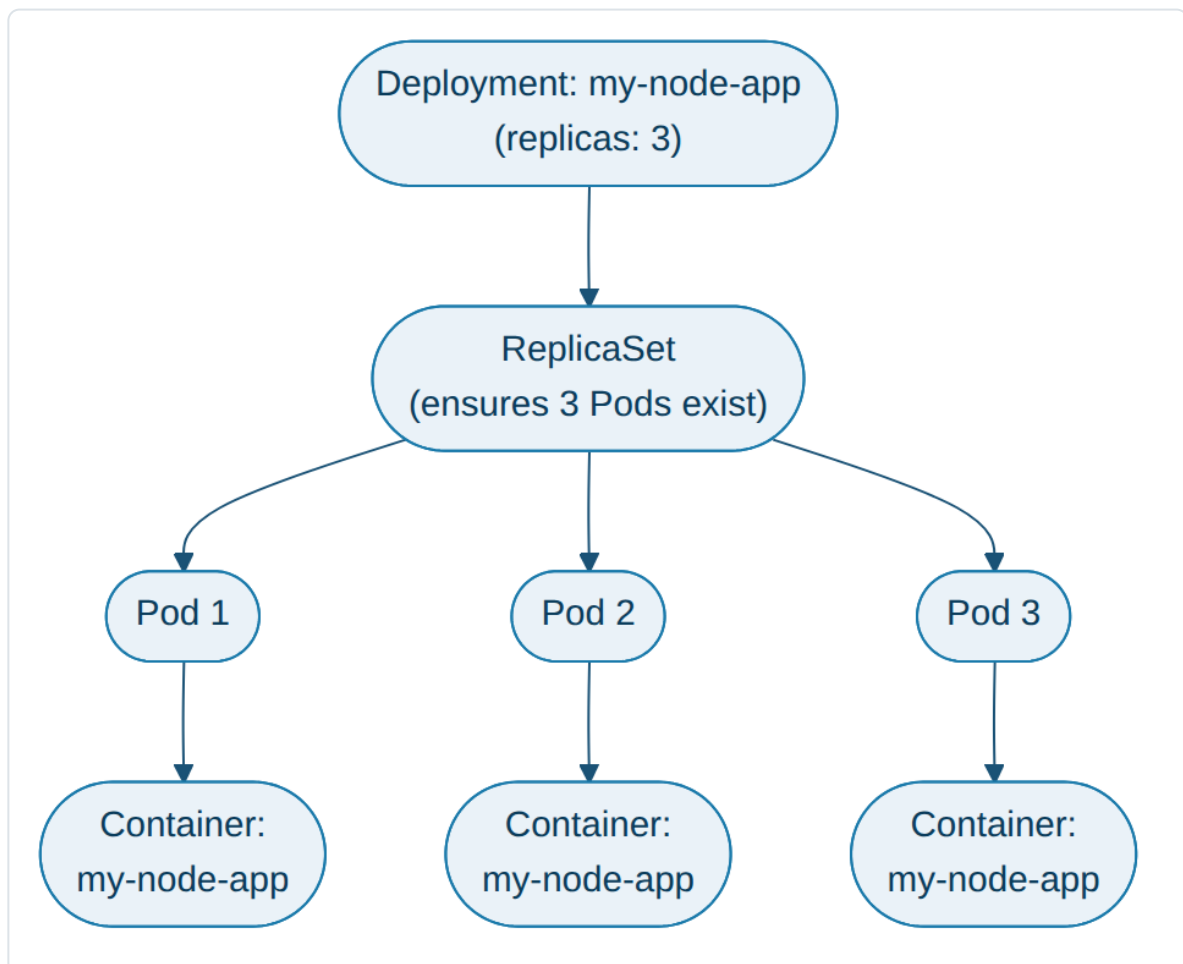


Kubernetes Core Objects

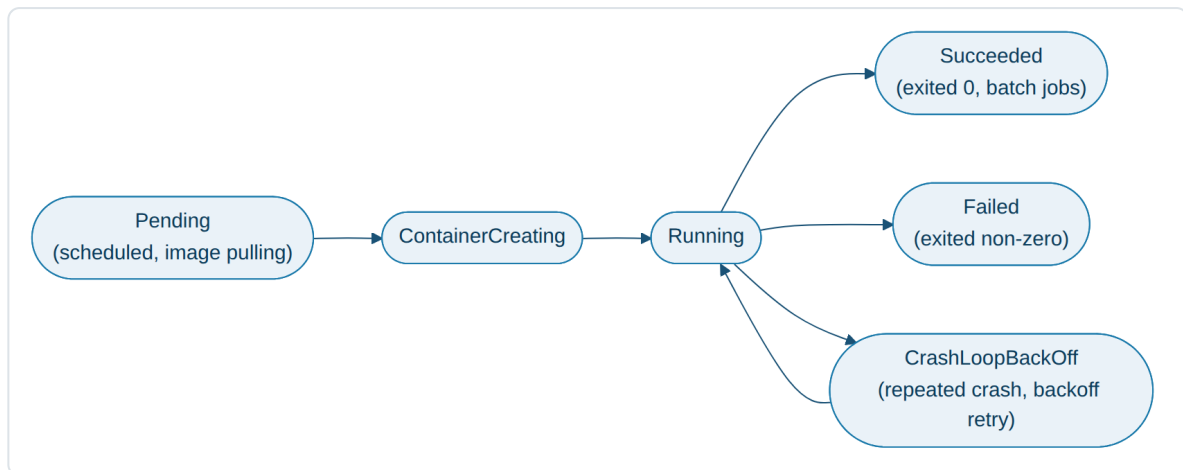
- **Pod** → the smallest deployable unit in Kubernetes — one or more tightly-coupled containers that share the same network namespace (same IP) & storage volumes. Usually 1 container per Pod.
- **ReplicaSet** → ensures a specified number of identical Pod replicas are running at all times; replaces a Pod automatically if it dies.
- **Deployment** → a higher-level object that manages ReplicaSets for you — handles rolling updates, rollbacks, and scaling declaratively (you almost always create a Deployment, not a bare ReplicaSet/ Pod).
- **Service** → a stable network endpoint (fixed IP/DNS name) that load-balances traffic across a dynamic, changing set of Pods (Pods get recreated with new IPs constantly — the Service hides that).
- **ClusterIP** (default) → reachable only from inside the cluster.
- **NodePort** → exposes the Service on a static port on every Node's IP.

- **LoadBalancer** → provisions an external cloud Load Balancer (on GKE, this creates a real GCP Load Balancer automatically).
- **Namespace** → a virtual sub-cluster used to logically separate resources (eg. `dev`, `staging`, `prod` namespaces in one physical cluster).
- **ConfigMap** → stores non-sensitive configuration (key-value pairs) separately from the container image, injected as env vars or files.
- **Secret** → same idea as ConfigMap, but for sensitive data (passwords, tokens) — base64-encoded & access-controlled.
- **Volume / PersistentVolume (PV) / PersistentVolumeClaim (PVC)** → provide storage that can outlive a single Pod's lifecycle.
- **Ingress** → manages external HTTP(S) access to Services inside the cluster, with host/path-based routing (similar role to a Load Balancer's URL Map).

Object hierarchy diagram



Pod lifecycle diagram



Rolling update flow (Deployment)

- When you update a Deployment's image version, Kubernetes doesn't kill all old Pods at once — it performs a **rolling update**: gradually spins up new-version Pods and terminates old-version Pods, keeping the app available throughout.



Service types diagram



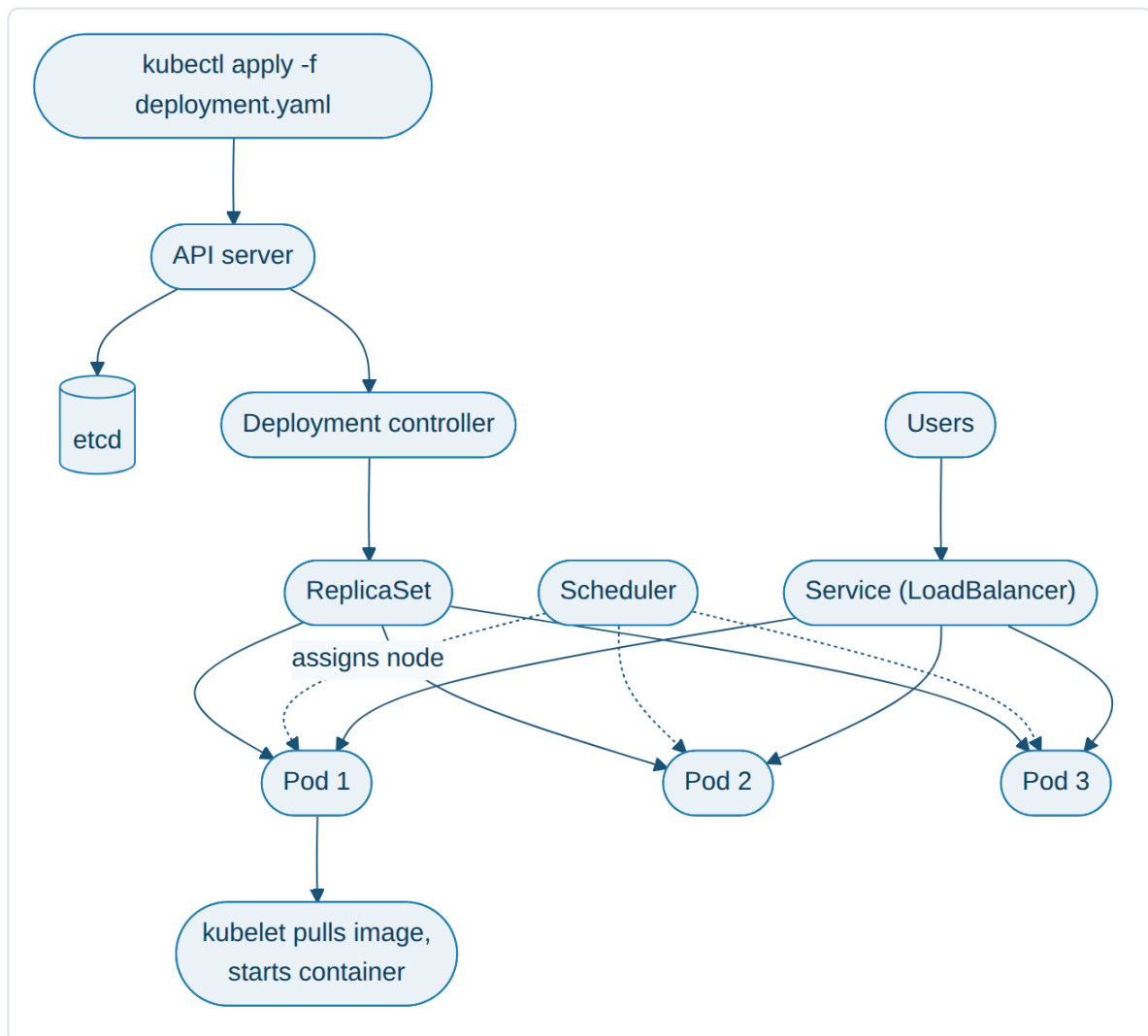
Example → Deploying a sample app with kubectl

```
# deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-node-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: my-node-app
  template:
    metadata:
      labels:
        app: my-node-app
    spec:
      containers:
        - name: my-node-app
          image: asia-south1-docker.pkg.dev/my-project/my-repo/my-node-app:
1.0
          ports:
            - containerPort: 3000
---
# service.yaml
apiVersion: v1
kind: Service
metadata:
  name: my-node-app-svc
spec:
  type: LoadBalancer
  selector:
    app: my-node-app
  ports:
    - port: 80
      targetPort: 3000
```

```
kubectl apply -f deployment.yaml
kubectl apply -f service.yaml
kubectl get pods
kubectl get service my-node-app-svc
```

1. `kubectl apply` sends the YAML to the **API server**.

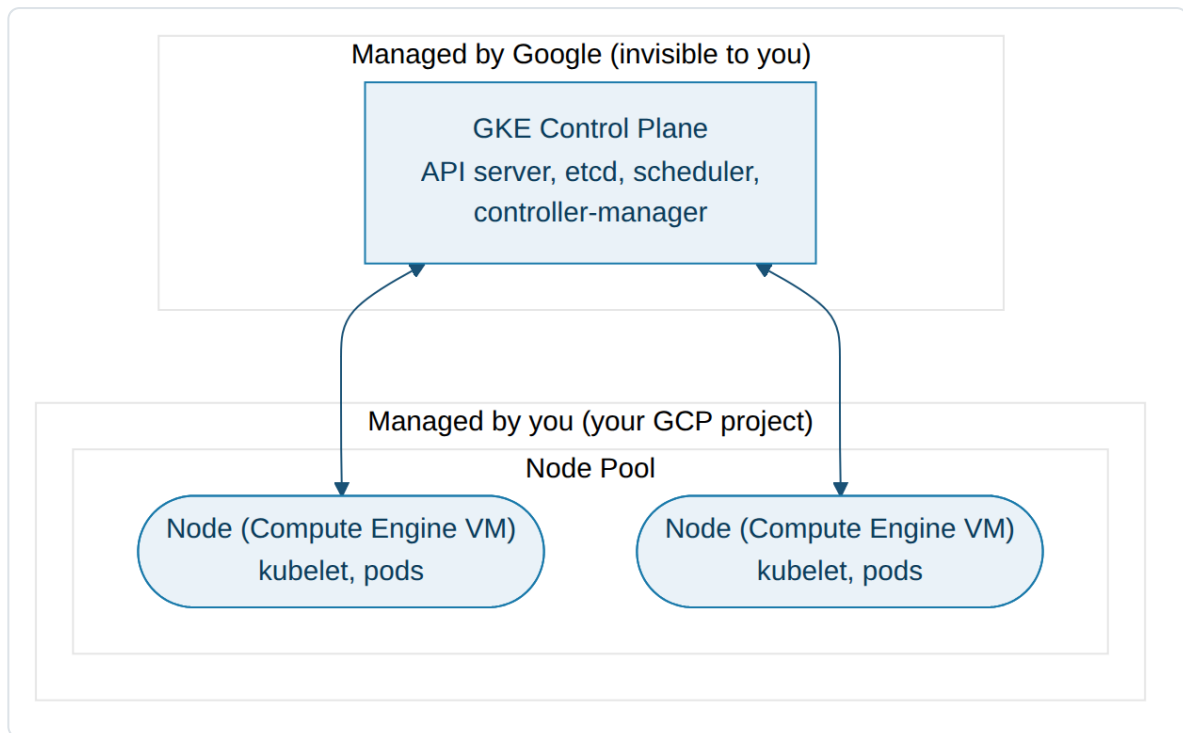
2. API server writes desired state to **etcd**.
3. **Deployment controller** creates a **ReplicaSet**, which creates 3 **Pods**.
4. **Scheduler** assigns each Pod to a Node.
5. Each Node's **kubelet** pulls the image (via the container runtime) and starts the containers.
6. The **Service** (`type: LoadBalancer`) gets a stable external IP and load-balances traffic across the 3 Pods, wherever they land.



GKE (Google Kubernetes Engine)

- GKE → Google's **fully-managed Kubernetes** service — you get a production-ready K8s cluster without installing/operating the control plane yourself.
- Google manages, patches & upgrades the **Control Plane** (API server, etcd, scheduler, controller-manager) for you — it's not even visible as a VM you can SSH into.
- You manage → your **workloads** (Deployments, Services, etc.) and, in Standard mode, your **Node Pools**.

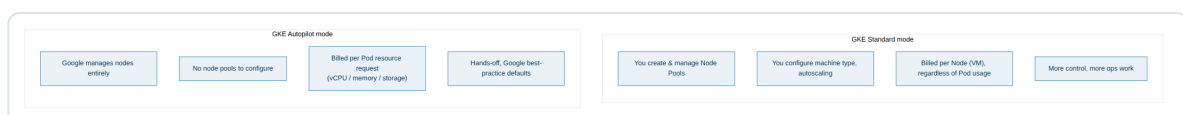
GKE architecture diagram



GKE modes → Autopilot vs Standard

- **Standard mode** → you choose & manage the Node Pools yourself (machine type, number of nodes, autoscaling settings); more control, but more ops work; billed per-node (per Compute Engine VM), regardless of Pod resource usage.
- **Autopilot mode** → Google manages the nodes entirely (fully hands-off, no node pools to configure); Google applies security/best-practice defaults automatically; billed per Pod resource request (vCPU/memory/storage actually requested), not per node — can be more cost-efficient for variable workloads.
- Recommendation → use **Autopilot** for most workloads by default; use **Standard** when you need special node-level privileges, custom node configuration, or GPUs/specialized hardware not supported in Autopilot.

Autopilot vs Standard diagram

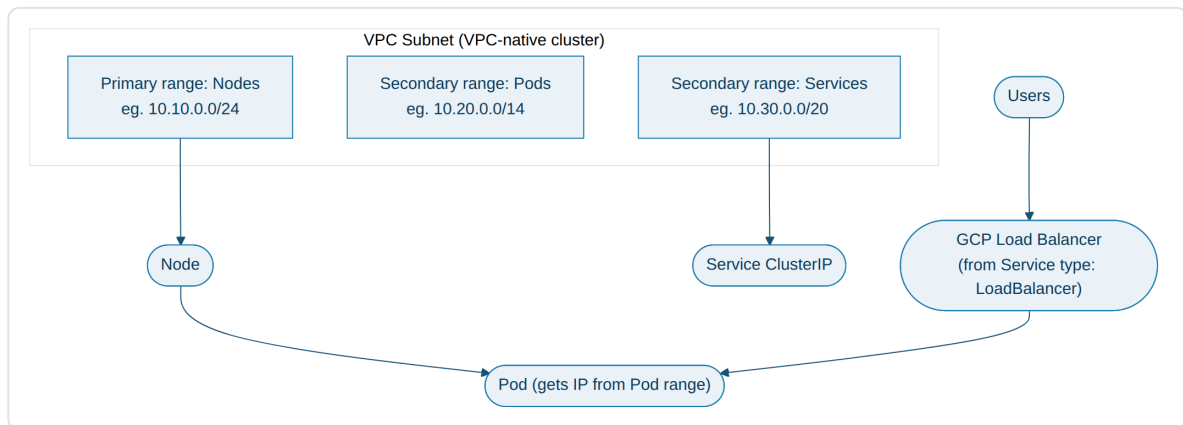


GKE Networking

- GKE clusters are typically **VPC-native** → every Pod gets a real, routable internal IP from a **secondary CIDR range** on the cluster's VPC subnet (recall Subnet & CIDR notes) — Pods can talk directly to other GCP resources without extra NAT hops.

- A GKE cluster uses (at least) 3 IP ranges from the VPC → one for Nodes (primary subnet range), one secondary range for Pods, one secondary range for Services (ClusterIPs).
- A **Service** of **type: LoadBalancer** in GKE automatically provisions a real **Google Cloud Load Balancer** pointing at the right Pods — tying directly back into the Load Balancer concepts from the GCP notes.

GKE networking diagram



Example → End-to-end CI/CD: code to running Pods on GKE

1. Developer pushes code to Git → CI pipeline triggers.
2. CI runs `docker build` → produces image `my-node-app:1.2`.
3. CI runs `docker push` → image lands in **Artifact Registry**.
4. CI runs `kubectl set image deployment/my-node-app my-node-app=...my-node-app:1.2` (or a GitOps/Cloud Deploy pipeline).
5. GKE's Deployment controller performs a **rolling update** → new Pods pull `1.2` from Artifact Registry, old `1.1` Pods are drained & removed.
6. The existing **LoadBalancer** Service keeps routing user traffic throughout — zero downtime.
7. End users hit the GCP Load Balancer's external IP → traffic reaches the new Pods, running the new code.

